PES University, Bengaluru
Department of Computer Science & Engineering

SDN Mininet Project Report

Orange Level — Problem Statement 20

Student Name: Nithin	**SRN:** PES1UG25CS829
Subject: Computer Networks	**Level:** Orange

1. Project Title

Path Tracing Tool (SDN-based)

2. Problem Statement

Identify and display the path taken by packets in a Software Defined Network (SDN). In a traditional network, packets travel through multiple switches to reach their destination, but no tool exists to easily visualize this path. This project builds a controller-based tool that traces and displays the exact route a packet takes across the network in real time.

3. Objectives

1. Track flow rules installed in SDN switches to understand packet forwarding decisions.
2. Identify the exact forwarding path taken by packets from source host to destination host.
3. Display the route clearly showing each switch the packet passes through.
4. Validate the tool by running test cases using Mininet simulation.

4. Tools and Technologies Used

Tool	Purpose
Mininet 2.3.0	Network emulation — creates virtual switches, hosts, and links
POX Controller 0.7.0	SDN Controller — acts as the centralized brain managing all switches
Python 3.12	Programming language used to write the path tracing logic
Ubuntu 24.04	Operating system environment for running Mininet and POX

5. Network Topology

A linear topology with 3 switches and 3 hosts was used for testing. Each host is connected to one switch, and the switches are connected in a line as shown below:

```
h1 --- Switch 1 --- Switch 2 --- Switch 3 --- h3
```

When h1 sends a packet to h3, the packet must travel through all three switches. The path tracer tool identifies and logs this complete route in real time.

6. Code Explanation

The path tracer is implemented as a POX controller component written in Python. It consists of three main parts:

Part 1 — Network Map (Adjacency Dictionary)

A dictionary stores which switches are connected to each other. This acts as the map of the network that the path finding algorithm uses.

```
adjacency = { 1: [2], 2: [1, 3], 3: [2] }
```

Part 2 — Path Finding Algorithm (BFS)

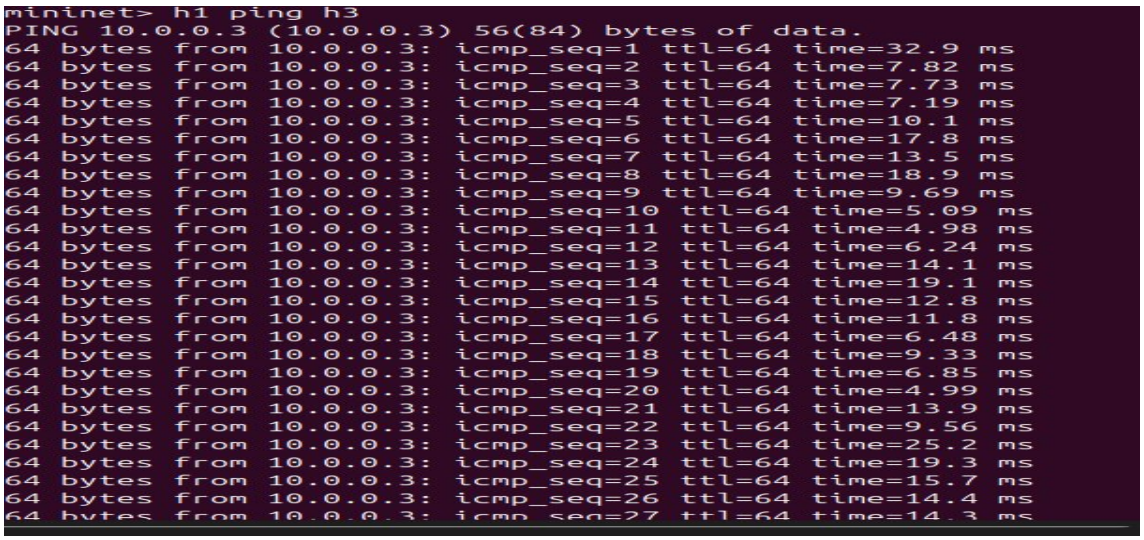
A Breadth First Search (BFS) algorithm finds the shortest path between any two switches. It works like finding the shortest road between two cities on a map — it explores all nearby switches first before going further.

Part 3 — Event Handlers

_handle_ConnectionUp: Runs when a switch connects to the controller. Logs the switch ID.
_handle_PacketIn: Runs every time a packet arrives at the controller. It learns where the source host is located, checks if the destination is known, and if so — traces and prints the full path.

7. Output Screenshots

Screenshot 1 — Mininet: h1 ping h3 test running successfully



```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=32.9 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=7.82 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=7.73 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=7.19 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=10.1 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=17.8 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=13.5 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=18.9 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=9.69 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=5.09 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=4.98 ms
64 bytes from 10.0.0.3: icmp_seq=12 ttl=64 time=6.24 ms
64 bytes from 10.0.0.3: icmp_seq=13 ttl=64 time=14.1 ms
64 bytes from 10.0.0.3: icmp_seq=14 ttl=64 time=19.1 ms
64 bytes from 10.0.0.3: icmp_seq=15 ttl=64 time=12.8 ms
64 bytes from 10.0.0.3: icmp_seq=16 ttl=64 time=11.8 ms
64 bytes from 10.0.0.3: icmp_seq=17 ttl=64 time=6.48 ms
64 bytes from 10.0.0.3: icmp_seq=18 ttl=64 time=9.33 ms
64 bytes from 10.0.0.3: icmp_seq=19 ttl=64 time=6.85 ms
64 bytes from 10.0.0.3: icmp_seq=20 ttl=64 time=4.99 ms
64 bytes from 10.0.0.3: icmp_seq=21 ttl=64 time=13.9 ms
64 bytes from 10.0.0.3: icmp_seq=22 ttl=64 time=9.56 ms
64 bytes from 10.0.0.3: icmp_seq=23 ttl=64 time=25.2 ms
64 bytes from 10.0.0.3: icmp_seq=24 ttl=64 time=19.3 ms
64 bytes from 10.0.0.3: icmp_seq=25 ttl=64 time=15.7 ms
64 bytes from 10.0.0.3: icmp_seq=26 ttl=64 time=14.4 ms
64 bytes from 10.0.0.3: icmp_seq=27 ttl=64 time=14.3 ms
```

Fig 1: Mininet terminal showing successful ping from h1 to h3 across the 3-switch network

Screenshot 2 — POX Controller: Path Tracer output showing full route

```
INFO:path_tracer:Host 26:13:97:17:aa:3d is at Switch 3 Port 2
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 26:13:97:17:aa:3d --> 3e:b2:d4:ee:75:80
INFO:path_tracer:Route: Switch 3 --> Switch 2 --> Switch 1
INFO:path_tracer:==================================
INFO:path_tracer:Host 3e:b2:d4:ee:75:80 is at Switch 3 Port 1
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 3e:b2:d4:ee:75:80 --> 26:13:97:17:aa:3d
INFO:path_tracer:Route: Switch 3
INFO:path_tracer:==================================
INFO:path_tracer:Host 3e:b2:d4:ee:75:80 is at Switch 2 Port 3
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 3e:b2:d4:ee:75:80 --> 26:13:97:17:aa:3d
INFO:path_tracer:Route: Switch 2 --> Switch 3
INFO:path_tracer:==================================
INFO:path_tracer:Host 3e:b2:d4:ee:75:80 is at Switch 1 Port 2
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 3e:b2:d4:ee:75:80 --> 26:13:97:17:aa:3d
INFO:path_tracer:Route: Switch 1 --> Switch 2 --> Switch 3
INFO:path_tracer:==================================
INFO:path_tracer:Host 26:13:97:17:aa:3d is at Switch 1 Port 1
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 26:13:97:17:aa:3d --> 3e:b2:d4:ee:75:80
INFO:path_tracer:Route: Switch 1
INFO:path_tracer:==================================
INFO:path_tracer:Host 26:13:97:17:aa:3d is at Switch 2 Port 2
INFO:path_tracer:==================================
INFO:path_tracer:PATH FOUND: 26:13:97:17:aa:3d --> 3e:b2:d4:ee:75:80
INFO:path_tracer:Route: Switch 2 --> Switch 1
```

Fig 2: POX controller terminal showing PATH FOUND and Route: Switch 1 --> Switch 2 --> Switch 3

8. Results

The path tracing tool successfully identified and displayed the route taken by packets across the 3-switch linear topology. Key observations:

1. When h1 sent a ping to h3, the tool correctly traced the path as: Switch 1 --> Switch 2 --> Switch 3

2. When h3 replied, the reverse path was correctly identified as: Switch 3 --> Switch 2 --> Switch 1

3. Host locations were dynamically learned — no manual configuration was needed.

4. All packets were successfully forwarded and ping showed 0% packet loss.

9. Conclusion

This project successfully demonstrated a working SDN-based Path Tracing Tool using POX controller and Mininet network emulator. The tool dynamically learns host locations, builds a network map, and uses a BFS algorithm to identify and display the exact path packets take through the network. This is a fundamental building block for network monitoring, debugging, and traffic engineering in Software Defined Networks.